

Advanced Python Programming

Unit 2 – GUI Programming | Assignment No. 1

Q1. Write a Python GUI program to create a label and change the label font style (font name, bold, size) using tkinter module.

Program:

```
import tkinter as tk
from tkinter import font

def change_font():
    selected_font = font_var.get()
    selected_size = int(size_var.get())
    bold_val = "bold" if bold_var.get() else "normal"
    label.config(font=(selected_font, selected_size, bold_val))

root = tk.Tk()
root.title("Label Font Changer")
root.geometry("420x280")

label = tk.Label(root, text="Hello, Tkinter!", font=("Arial", 14))
label.pack(pady=20)

frame = tk.Frame(root)
frame.pack()

tk.Label(frame, text="Font:").grid(row=0, column=0, padx=5)
font_var = tk.StringVar(value="Arial")
fonts = ["Arial", "Times New Roman", "Courier New", "Verdana"]
tk.OptionMenu(frame, font_var, *fonts).grid(row=0, column=1, padx=5)

tk.Label(frame, text="Size:").grid(row=0, column=2, padx=5)
size_var = tk.StringVar(value="14")
tk.Spinbox(frame, from_=8, to=40, textvariable=size_var, width=5).grid(row=0, column=3)

bold_var = tk.BooleanVar()
tk.Checkbutton(root, text="Bold", variable=bold_var).pack(pady=5)

tk.Button(root, text="Apply Font", command=change_font,
          bg="#1a237e", fg="white").pack(pady=8)

root.mainloop()
```

Output Description:

➤ **Output:** A window appears with a label 'Hello, Tkinter!'. User can choose font name, size, toggle bold and click 'Apply Font' to update the label style dynamically.

Q2. Write a Python program that creates a Tkinter window with a Frame widget containing multiple labels, Entry widgets, and buttons. Put these widgets inside the Frame using the Place geometry manager.

Program:

```
import tkinter as tk
from tkinter import messagebox

def submit():
```

```

name = name_entry.get()
email = email_entry.get()
if name and email:
    messagebox.showinfo("Submitted", f"Name: {name}\nEmail: {email}")
else:
    messagebox.showwarning("Warning", "Please fill all fields!")

root = tk.Tk()
root.title("Place Geometry Manager Demo")
root.geometry("400x300")

frame = tk.Frame(root, bg="#e8eaf6", bd=2, relief="groove")
frame.place(x=30, y=30, width=340, height=230)

tk.Label(frame, text="User Registration Form",
         font=("Arial", 13, "bold"), bg="#e8eaf6",
         fg="#1a237e").place(x=70, y=10)

tk.Label(frame, text="Name:", bg="#e8eaf6").place(x=20, y=60)
name_entry = tk.Entry(frame, width=25)
name_entry.place(x=120, y=60)

tk.Label(frame, text="Email:", bg="#e8eaf6").place(x=20, y=100)
email_entry = tk.Entry(frame, width=25)
email_entry.place(x=120, y=100)

tk.Label(frame, text="Phone:", bg="#e8eaf6").place(x=20, y=140)
phone_entry = tk.Entry(frame, width=25)
phone_entry.place(x=120, y=140)

tk.Button(frame, text="Submit", command=submit,
         bg="#1a237e", fg="white", width=10).place(x=80, y=185)
tk.Button(frame, text="Clear",
         command=lambda: [name_entry.delete(0, 'end'),
                          email_entry.delete(0, 'end'),
                          phone_entry.delete(0, 'end')],
         bg="#c62828", fg="white", width=10).place(x=190, y=185)

root.mainloop()

```

Output Description:

► **Output: A window with a Frame using Place manager. Labels, Entry fields for Name/Email/Phone, and Submit/Clear buttons are positioned precisely using x,y coordinates.**

Q3. Write a Python program that builds an employee list application with a Listbox widget to display names. Use the Pack geometry manager to align the Listbox and buttons.

Program:

```

import tkinter as tk
from tkinter import messagebox, simpledialog

root = tk.Tk()
root.title("Employee List Application")
root.geometry("380x400")

tk.Label(root, text="Employee List",
         font=("Arial", 14, "bold"), fg="#1a237e").pack(pady=8)

```

```

frame_list = tk.Frame(root)
frame_list.pack(pady=5)

scrollbar = tk.Scrollbar(frame_list)
scrollbar.pack(side=tk.RIGHT, fill=tk.Y)

listbox = tk.Listbox(frame_list, width=35, height=12,
                    yscrollcommand=scrollbar.set,
                    font=("Arial", 11), selectbackground="#3949ab")
listbox.pack(side=tk.LEFT)
scrollbar.config(command=listbox.yview)

employees = ["Alice Johnson", "Bob Smith", "Carol White",
            "David Brown", "Eva Green"]
for emp in employees:
    listbox.insert(tk.END, emp)

def add_employee():
    name = simpledialog.askstring("Add Employee", "Enter employee name:")
    if name:
        listbox.insert(tk.END, name)

def delete_employee():
    selected = listbox.curselection()
    if selected:
        listbox.delete(selected)
    else:
        messagebox.showwarning("Warning", "Please select an employee!")

def show_employee():
    selected = listbox.curselection()
    if selected:
        messagebox.showinfo("Employee", listbox.get(selected))
    else:
        messagebox.showwarning("Warning", "Please select an employee!")

btn_frame = tk.Frame(root)
btn_frame.pack(pady=10)

tk.Button(btn_frame, text="Add", command=add_employee,
          bg="#2e7d32", fg="white", width=10).pack(side=tk.LEFT, padx=5)
tk.Button(btn_frame, text="Delete", command=delete_employee,
          bg="#c62828", fg="white", width=10).pack(side=tk.LEFT, padx=5)
tk.Button(btn_frame, text="View", command=show_employee,
          bg="#1565c0", fg="white", width=10).pack(side=tk.LEFT, padx=5)

root.mainloop()

```

Output Description:

➤ **Output:** A window showing a scrollable Listbox with employee names. Pack manager aligns all widgets vertically. Add, Delete, View buttons manage the list.

Q4. Write a Python program that implements a contact information form with labels and Entry widgets. Organize them within a Frame widget using the Place geometry manager.

Program:

```
import tkinter as tk
```

```

from tkinter import messagebox

def save_contact():
    data = {
        "Name" : name_e.get(),
        "Phone" : phone_e.get(),
        "Email" : email_e.get(),
        "Address": addr_e.get(),
        "City" : city_e.get(),
    }
    if all(data.values()):
        info = "\n".join([f"{k}: {v}" for k, v in data.items()])
        messagebox.showinfo("Contact Saved", info)
    else:
        messagebox.showwarning("Incomplete", "Please fill all fields.")

root = tk.Tk()
root.title("Contact Information Form")
root.geometry("450x350")
root.config(bg="#f5f5f5")

frame = tk.Frame(root, bg="white", bd=2, relief="ridge")
frame.place(x=25, y=20, width=400, height=300)

tk.Label(frame, text="Contact Information",
        font=("Arial", 13, "bold"), bg="white",
        fg="#1a237e").place(x=110, y=10)

fields = [("Full Name:", 50), ("Phone No.:", 90),
        ("Email ID:", 130), ("Address:", 170), ("City:", 210)]

entries = []
for (lbl, y) in fields:
    tk.Label(frame, text=lbl, bg="white",
            font=("Arial", 10)).place(x=15, y=y)
    e = tk.Entry(frame, width=28, font=("Arial", 10))
    e.place(x=130, y=y)
    entries.append(e)

name_e, phone_e, email_e, addr_e, city_e = entries

tk.Button(frame, text="Save Contact", command=save_contact,
        bg="#1a237e", fg="white", font=("Arial", 10),
        width=14).place(x=130, y=255)

root.mainloop()

```

Output Description:

➤ **Output:** A neatly laid contact form inside a Frame. Labels and Entry widgets placed using Place manager at exact coordinates. Save Contact shows all information in a messagebox.

Q5. Write a Python GUI program to create a Menu bar with File, Edit, and Help menus, each containing submenu items using tkinter module.

Program:

```

import tkinter as tk
from tkinter import messagebox, filedialog

```

```

root = tk.Tk()
root.title("Menu Bar Demo")
root.geometry("500x350")

text_area = tk.Text(root, font=("Arial", 11))
text_area.pack(expand=True, fill="both", padx=10, pady=10)

menubar = tk.Menu(root)

# --- File Menu ---
file_menu = tk.Menu(menubar, tearoff=0)
file_menu.add_command(label="New",
    command=lambda: text_area.delete("1.0", tk.END))
file_menu.add_command(label="Open",
    command=lambda: text_area.insert(tk.END,
        open(filedialog.askopenfilename()).read()
        if filedialog.askopenfilename() else ""))
file_menu.add_command(label="Save",
    command=lambda: messagebox.showinfo("Save", "File Saved!"))
file_menu.add_separator()
file_menu.add_command(label="Exit", command=root.quit)
menubar.add_cascade(label="File", menu=file_menu)

# --- Edit Menu ---
edit_menu = tk.Menu(menubar, tearoff=0)
edit_menu.add_command(label="Cut",
    command=lambda: root.event_generate("<<Cut>>"))
edit_menu.add_command(label="Copy",
    command=lambda: root.event_generate("<<Copy>>"))
edit_menu.add_command(label="Paste",
    command=lambda: root.event_generate("<<Paste>>"))
edit_menu.add_separator()
edit_menu.add_command(label="Select All",
    command=lambda: text_area.tag_add("sel", "1.0", tk.END))
menubar.add_cascade(label="Edit", menu=edit_menu)

# --- Help Menu ---
help_menu = tk.Menu(menubar, tearoff=0)
help_menu.add_command(label="About",
    command=lambda: messagebox.showinfo("About",
        "Advanced Python GUI\nUnit 2 Assignment"))
help_menu.add_command(label="Documentation",
    command=lambda: messagebox.showinfo("Docs",
        "Visit: https://docs.python.org/tkinter"))
menubar.add_cascade(label="Help", menu=help_menu)

root.config(menu=menubar)
root.mainloop()

```

Output Description:

► **Output:** A text editor-style window with File, Edit, Help menus. File menu has New/Open/Save/Exit, Edit has Cut/Copy/Paste/Select All, Help has About and Documentation.

Q6. Write a Python GUI program to create a Colorchooser widget to select a color using tkinter module.

Program:

```

import tkinter as tk
from tkinter import colorchooser

def choose_bg_color():
    color = colorchooser.askcolor(title="Choose Background Color")
    if color[1]:
        canvas.config(bg=color[1])
        bg_label.config(text=f"BG Color: {color[1]}", bg=color[1])

def choose_text_color():
    color = colorchooser.askcolor(title="Choose Text Color")
    if color[1]:
        sample_label.config(fg=color[1])
        fg_label.config(text=f"Text Color: {color[1]}")

root = tk.Tk()
root.title("Color Chooser Demo")
root.geometry("420x320")

canvas = tk.Canvas(root, bg="ffffff", height=150, width=380)
canvas.pack(padx=20, pady=10)

sample_label = tk.Label(canvas, text="Sample Text - Color Preview",
                        font=("Arial", 14, "bold"), bg="ffffff")
sample_label.place(x=50, y=60)

btn_frame = tk.Frame(root)
btn_frame.pack()

tk.Button(btn_frame, text="Choose BG Color",
          command=choose_bg_color, bg="#283593",
          fg="white", width=18).pack(side=tk.LEFT, padx=8, pady=5)
tk.Button(btn_frame, text="Choose Text Color",
          command=choose_text_color, bg="#b71c1c",
          fg="white", width=18).pack(side=tk.LEFT, padx=8, pady=5)

bg_label = tk.Label(root, text="BG Color: #ffffff", font=("Courier", 9))
bg_label.pack()
fg_label = tk.Label(root, text="Text Color: (default)", font=("Courier", 9))
fg_label.pack()

root.mainloop()

```

Output Description:

➤ **Output:** Window with a canvas area showing sample text. Two buttons open the system color picker. Chosen BG color updates canvas background; Text color updates sample text color.

Q7. Write a Python program that allows the user to draw and manipulate geometric shapes (e.g., rectangle, circle) on the 'Canvas' using Tkinter.

Program:

```

import tkinter as tk

root = tk.Tk()
root.title("Shape Drawing on Canvas")
root.geometry("600x480")

```

```

shape_var = tk.StringVar(value="Rectangle")
color_var = tk.StringVar(value="blue")
shapes = []
selected = [None]
start_x = [0]; start_y = [0]

canvas = tk.Canvas(root, bg="white", width=580, height=380,
                    relief="sunken", bd=2)
canvas.pack(padx=10, pady=5)

ctrl = tk.Frame(root)
ctrl.pack()

for s in ["Rectangle", "Oval", "Line"]:
    tk.Radiobutton(ctrl, text=s, variable=shape_var,
                   value=s).pack(side=tk.LEFT, padx=6)

for c in ["blue", "red", "green", "black", "orange"]:
    tk.Button(ctrl, bg=c, width=2,
              command=lambda col=c: color_var.set(col)).pack(side=tk.LEFT, padx=2)

tk.Button(ctrl, text="Clear All",
          command=lambda: canvas.delete("all"),
          bg="#c62828", fg="white").pack(side=tk.LEFT, padx=8)

def on_press(event):
    start_x[0] = event.x
    start_y[0] = event.y

def on_release(event):
    x0, y0 = start_x[0], start_y[0]
    x1, y1 = event.x, event.y
    col = color_var.get()
    sh = shape_var.get()
    if sh == "Rectangle":
        canvas.create_rectangle(x0, y0, x1, y1,
                               outline=col, width=2)
    elif sh == "Oval":
        canvas.create_oval(x0, y0, x1, y1,
                           outline=col, width=2)
    elif sh == "Line":
        canvas.create_line(x0, y0, x1, y1,
                           fill=col, width=2)

canvas.bind("<ButtonPress-1>", on_press)
canvas.bind("<ButtonRelease-1>", on_release)

root.mainloop()

```

Output Description:

► **Output:** Canvas where user selects shape type (Rectangle/Oval/Line) and color. Click and drag on canvas to draw the selected shape. 'Clear All' removes all shapes.

Q8. Write a Python program that implements a temperature converter application using Tkinter, allowing users to convert between Celsius and Fahrenheit.

Program:

```

import tkinter as tk
from tkinter import messagebox

def to_fahrenheit():
    try:
        c = float(celsius_entry.get())
        f = (c * 9/5) + 32
        result_label.config(
            text=f"{c}°C = {f:.2f}°F",
            fg="#1a237e")
    except ValueError:
        messagebox.showerror("Error", "Enter a valid number!")

def to_celsius():
    try:
        f = float(fahr_entry.get())
        c = (f - 32) * 5/9
        result_label.config(
            text=f"{f}°F = {c:.2f}°C",
            fg="#b71c1c")
    except ValueError:
        messagebox.showerror("Error", "Enter a valid number!")

root = tk.Tk()
root.title("Temperature Converter")
root.geometry("400x320")
root.config(bg="#e8eaf6")

tk.Label(root, text="Temperature Converter",
        font=("Arial", 15, "bold"),
        bg="#e8eaf6", fg="#1a237e").pack(pady=12)

frame1 = tk.LabelFrame(root, text="Celsius → Fahrenheit",
        bg="#e8eaf6", font=("Arial", 10))
frame1.pack(padx=20, pady=6, fill="x")
tk.Label(frame1, text="Celsius:", bg="#e8eaf6").grid(row=0, column=0, padx=8, pady=6)
celsius_entry = tk.Entry(frame1, width=15, font=("Arial", 11))
celsius_entry.grid(row=0, column=1, padx=8)
tk.Button(frame1, text="Convert →", command=to_fahrenheit,
        bg="#1a237e", fg="white").grid(row=0, column=2, padx=8)

frame2 = tk.LabelFrame(root, text="Fahrenheit → Celsius",
        bg="#e8eaf6", font=("Arial", 10))
frame2.pack(padx=20, pady=6, fill="x")
tk.Label(frame2, text="Fahrenheit:", bg="#e8eaf6").grid(row=0, column=0, padx=8, pady=6)
fahr_entry = tk.Entry(frame2, width=15, font=("Arial", 11))
fahr_entry.grid(row=0, column=1, padx=8)
tk.Button(frame2, text="Convert →", command=to_celsius,
        bg="#b71c1c", fg="white").grid(row=0, column=2, padx=8)

result_label = tk.Label(root, text="Result will appear here",
        font=("Arial", 13, "bold"),
        bg="#e8eaf6", fg="#2e7d32")
result_label.pack(pady=15)

root.mainloop()

```


Output Description:

► **Output: Two sections – Celsius-to-Fahrenheit and Fahrenheit-to-Celsius. Enter a value, click Convert, and the result displays below with correct formula applied.**

Q9. Write a Python program to create a Tkinter-based login form with input fields for userid and password.

Program:

```
import tkinter as tk
from tkinter import messagebox

USERS = {"admin": "admin123", "student": "pass456"}

def login():
    uid = user_entry.get()
    pwd = pass_entry.get()
    if not uid or not pwd:
        messagebox.showwarning("Incomplete", "Please enter both fields.")
        return
    if uid in USERS and USERS[uid] == pwd:
        messagebox.showinfo("Login Successful",
                             f"Welcome, {uid}!")
        root.destroy()
    else:
        messagebox.showerror("Login Failed",
                             "Invalid User ID or Password!")
        pass_entry.delete(0, tk.END)

def toggle_password():
    if pass_entry.cget("show") == "":
        pass_entry.config(show="")
        show_btn.config(text="Hide")
    else:
        pass_entry.config(show="*")
        show_btn.config(text="Show")

root = tk.Tk()
root.title("Login Form")
root.geometry("380x280")
root.config(bg="#1a237e")
root.resizable(False, False)

card = tk.Frame(root, bg="white", bd=0)
card.place(x=40, y=30, width=300, height=210)

tk.Label(card, text="USER LOGIN", font=("Arial", 14, "bold"),
         bg="white", fg="#1a237e").place(x=80, y=15)

tk.Label(card, text="User ID:", bg="white",
         font=("Arial", 10)).place(x=20, y=60)
user_entry = tk.Entry(card, width=22, font=("Arial", 11))
user_entry.place(x=110, y=58)

tk.Label(card, text="Password:", bg="white",
         font=("Arial", 10)).place(x=20, y=100)
pass_entry = tk.Entry(card, width=17, show="*", font=("Arial", 11))
```

```

pass_entry.place(x=110, y=98)
show_btn = tk.Button(card, text="Show", command=toggle_password,
                     font=("Arial", 8), width=4)
show_btn.place(x=258, y=98)

tk.Button(card, text="LOGIN", command=login,
         bg="#1a237e", fg="white",
         font=("Arial", 11, "bold"), width=16).place(x=75, y=150)

root.mainloop()

```

Output Description:

➤ **Output: A styled login form with dark blue background and white card. Credentials: admin/admin123 or student/pass456. Show/Hide toggles password visibility. Wrong credentials show error.**

Q10. Write a Python program that displays a button using Tkinter. When the button is clicked, display a message box with the message.

Program:

```

import tkinter as tk
from tkinter import messagebox

def show_message():
    messagebox.showinfo("Hello!", "Welcome to Advanced Python Programming!\nUnit 2: GUI with Tkinter")

def show_warning():
    messagebox.showwarning("Warning!", "This is a warning message box!")

def show_error():
    messagebox.showerror("Error!", "This is an error message box!")

def ask_question():
    ans = messagebox.askyesno("Question", "Do you like Python GUI?")
    if ans:
        result_lbl.config(text="Great! Python GUI is awesome!", fg="green")
    else:
        result_lbl.config(text="Give it another try!", fg="red")

root = tk.Tk()
root.title("Message Box Demo")
root.geometry("380x300")
root.config(bg="#e8eaf6")

tk.Label(root, text="Message Box Examples",
        font=("Arial", 14, "bold"),
        bg="#e8eaf6", fg="#1a237e").pack(pady=15)

buttons = [
    ("Show Info Message", show_message, "#1565c0"),
    ("Show Warning Message", show_warning, "#e65100"),
    ("Show Error Message", show_error, "#b71c1c"),
    ("Ask Yes/No Question", ask_question, "#2e7d32"),
]

for (txt, cmd, col) in buttons:
    tk.Button(root, text=txt, command=cmd,

```

```

        bg=col, fg="white",
        font=("Arial", 10), width=22).pack(pady=4)

result_lbl = tk.Label(root, text="", bg="#e8eaf6",
                      font=("Arial", 11, "italic"))
result_lbl.pack(pady=8)

root.mainloop()

```

Output Description:

► **Output:** Four buttons demonstrating all types of Tkinter message boxes – Info, Warning, Error, and Yes/No dialog. Yes/No response updates a label on the main window.

Q11. Write a Python program that implements a simple calculator application using Tkinter with buttons for digits (0-9) and arithmetic operators (+, -, *, /). Implement event handling to calculate.

Program:

```

import tkinter as tk
from tkinter import messagebox

expression = ""

def press(key):
    global expression
    if key == "=":
        try:
            result = str(eval(expression))
            display_var.set(result)
            expression = result
        except ZeroDivisionError:
            messagebox.showerror("Error", "Division by zero!")
            expression = ""
            display_var.set("0")
        except:
            messagebox.showerror("Error", "Invalid Expression!")
            expression = ""
            display_var.set("0")
    elif key == "C":
        expression = ""
        display_var.set("0")
    elif key == "⬅":
        expression = expression[:-1]
        display_var.set(expression if expression else "0")
    else:
        expression += str(key)
        display_var.set(expression)

root = tk.Tk()
root.title("Calculator")
root.geometry("320x430")
root.resizable(False, False)
root.config(bg="#263238")

display_var = tk.StringVar(value="0")
display = tk.Entry(root, textvariable=display_var, font=("Arial", 22),
                  justify="right", bd=0, bg="#37474f", fg="white",

```

```

        disabledbackground="#37474f", disabledforeground="white")
display.pack(fill="x", padx=10, pady=10, ipady=12)

buttons = [
    ["C", "■", "%", "/"],
    ["7", "8", "9", "*"],
    ["4", "5", "6", "-"],
    ["1", "2", "3", "+"],
    ["0", ".", "", "="],
]

colors = {"/": "#546e7a", "*": "#546e7a", "-": "#546e7a", "+": "#546e7a",
          "=": "#1565c0", "C": "#b71c1c", "■": "#e65100",
          "%": "#546e7a", ".": "#455a64"}

btn_frame = tk.Frame(root, bg="#263238")
btn_frame.pack(fill="both", expand=True, padx=10, pady=5)

for r, row in enumerate(buttons):
    for c, key in enumerate(row):
        if key == "":
            continue
        colspan = 2 if key == "0" else 1
        col = colors.get(key, "#455a64")
        b = tk.Button(btn_frame, text=key,
                      font=("Arial", 14, "bold"),
                      bg=col, fg="white",
                      activebackground="#78909c",
                      relief="flat", bd=0,
                      command=lambda k=key: press(k))
        b.grid(row=r, column=c, columnspan=colspan,
              sticky="nsew", padx=2, pady=2, ipady=10)

for i in range(5):
    btn_frame.rowconfigure(i, weight=1)
for i in range(4):
    btn_frame.columnconfigure(i, weight=1)

root.mainloop()

```

Output Description:

➤ **Output:** A fully functional dark-themed calculator with digit buttons (0-9), operators (+, -, *, /), decimal, clear (C), backspace, and equals (=) with proper expression evaluation.

Q12. Write a program to draw different shapes.

Program:

```

import tkinter as tk

root = tk.Tk()
root.title("Drawing Different Shapes")
root.geometry("600x480")

canvas = tk.Canvas(root, width=580, height=420,
                   bg="white", relief="sunken", bd=2)
canvas.pack(padx=10, pady=10)

```

```

# 1. Rectangle
canvas.create_rectangle(30, 30, 150, 100,
    fill="#bbdefb", outline="#1565c0", width=2)
canvas.create_text(90, 115, text="Rectangle",
    font=("Arial", 9), fill="#1565c0")

# 2. Oval / Ellipse
canvas.create_oval(180, 30, 330, 110,
    fill="#c8e6c9", outline="#2e7d32", width=2)
canvas.create_text(255, 120, text="Oval",
    font=("Arial", 9), fill="#2e7d32")

# 3. Circle
canvas.create_oval(370, 30, 470, 130,
    fill="#ffe0b2", outline="#e65100", width=2)
canvas.create_text(420, 140, text="Circle",
    font=("Arial", 9), fill="#e65100")

# 4. Line
canvas.create_line(30, 175, 200, 175,
    fill="#c62828", width=3)
canvas.create_text(115, 190, text="Line",
    font=("Arial", 9), fill="#c62828")

# 5. Triangle (Polygon)
canvas.create_polygon(260, 155, 210, 250, 310, 250,
    fill="#e1bee7", outline="#6a1b9a", width=2)
canvas.create_text(260, 265, text="Triangle",
    font=("Arial", 9), fill="#6a1b9a")

# 6. Pentagon
import math
cx, cy, r = 415, 200, 55
pts = [(cx + r*math.cos(math.radians(90 - 72*i)),
    cy - r*math.sin(math.radians(90 - 72*i)))
    for i in range(5)]
canvas.create_polygon(pts, fill="#fff9c4",
    outline="#f57f17", width=2)
canvas.create_text(415, 265, text="Pentagon",
    font=("Arial", 9), fill="#f57f17")

# 7. Arc
canvas.create_arc(30, 295, 180, 395,
    start=0, extent=270,
    fill="#b3e5fc", outline="#0277bd", width=2)
canvas.create_text(105, 405, text="Arc",
    font=("Arial", 9), fill="#0277bd")

# 8. Star (Polygon)
def star_points(cx, cy, r_out, r_in, n=5):
    pts = []
    for i in range(2*n):
        r = r_out if i % 2 == 0 else r_in
        angle = math.radians(90 - i * 180/n)
        pts.extend([cx + r*math.cos(angle),
            cy - r*math.sin(angle)])
    return pts

```

```

canvas.create_polygon(star_points(290, 340, 55, 25),
    fill="#ffcc80", outline="#e65100", width=2)
canvas.create_text(290, 405, text="Star",
    font=("Arial", 9), fill="#e65100")

# 9. Diamond
canvas.create_polygon(430,295, 490,345, 430,395, 370,345,
    fill="#f8bbd0", outline="#880e4f", width=2)
canvas.create_text(430, 408, text="Diamond",
    font=("Arial", 9), fill="#880e4f")

root.mainloop()

```

Output Description:

► **Output: Canvas displaying 9 geometric shapes: Rectangle, Oval, Circle, Line, Triangle, Pentagon, Arc, Star, and Diamond – each filled with different colors and labeled.**

Q13. Build a simple application with Tkinter that includes widgets such as labels, buttons, entry fields, and text areas. Arrange widgets using different geometry managers (pack, grid, place). Validate user input and provide feedback for invalid entries using message boxes or labels.

Program:

```

import tkinter as tk
from tkinter import messagebox
import re

root = tk.Tk()
root.title("Student Registration - Multi Geometry Manager")
root.geometry("520x560")
root.config(bg="#e8eaf6")

# ■■■ SECTION 1: PACK manager ■■■ title area
tk.Label(root, text="Student Registration Form",
    font=("Arial", 15, "bold"),
    bg="#1a237e", fg="white").pack(fill="x", pady=0, ipady=8)

info_label = tk.Label(root, text="",
    font=("Arial", 9, "italic"),
    bg="#e8eaf6", fg="red")
info_label.pack(pady=2)

# ■■■ SECTION 2: GRID manager ■■■ form fields
form_frame = tk.LabelFrame(root, text="Personal Details",
    bg="#e8eaf6", font=("Arial", 10, "bold"))
form_frame.pack(padx=20, pady=5, fill="x")

labels = ["Full Name*", "Email ID*", "Phone No.*",
    "Age*", "Department"]
entries = {}

for i, lbl in enumerate(labels):
    tk.Label(form_frame, text=lbl, bg="#e8eaf6",
        font=("Arial", 10)).grid(row=i, column=0,
            sticky="w", padx=12, pady=5)
    e = tk.Entry(form_frame, width=30, font=("Arial", 10))
    e.grid(row=i, column=1, padx=12, pady=5)
    entries[lbl] = e

```

```

# ■■ SECTION 3: PLACE manager ■■ text area + buttons
notes_frame = tk.LabelFrame(root, text="Additional Notes",
    bg="#e8eaf6", font=("Arial", 10, "bold"))
notes_frame.pack(padx=20, pady=5, fill="x")
notes_frame.config(height=100)

notes_text = tk.Text(notes_frame, height=4,
    font=("Arial", 10), width=50)
notes_text.pack(padx=8, pady=5)

def validate_and_submit():
    name = entries["Full Name*"].get().strip()
    email = entries["Email ID*"].get().strip()
    phone = entries["Phone No.*"].get().strip()
    age = entries["Age*"].get().strip()

    if not all([name, email, phone, age]):
        info_label.config(text="* All required fields must be filled!", fg="red")
        return

    if not re.match(r"^[^@]+@[^@]+\.[^@]+$", email):
        info_label.config(text="* Invalid email format!", fg="red")
        messagebox.showerror("Validation Error", "Please enter a valid email!")
        return

    if not phone.isdigit() or len(phone) != 10:
        info_label.config(text="* Phone must be 10 digits!", fg="red")
        messagebox.showerror("Validation Error",
            "Phone number must be exactly 10 digits!")
        return

    if not age.isdigit() or not (15 <= int(age) <= 80):
        info_label.config(text="* Age must be between 15 and 80!", fg="red")
        messagebox.showerror("Validation Error", "Age must be between 15 and 80!")
        return

    info_label.config(text="Form submitted successfully!", fg="green")
    messagebox.showinfo("Success",
        f"Registration Successful!\nName: {name}\nEmail: {email}")

def clear_form():
    for e in entries.values():
        e.delete(0, tk.END)
    notes_text.delete("1.0", tk.END)
    info_label.config(text="")

btn_frame = tk.Frame(root, bg="#e8eaf6")
btn_frame.pack(pady=10)
tk.Button(btn_frame, text="Submit", command=validate_and_submit,
    bg="#1a237e", fg="white", font=("Arial", 11), width=14).pack(side=tk.LEFT, padx=10)
tk.Button(btn_frame, text="Clear All", command=clear_form,
    bg="#c62828", fg="white", font=("Arial", 11), width=14).pack(side=tk.LEFT, padx=10)

root.mainloop()

```

Output Description:

➤ **Output:** Registration form using all three geometry managers: Pack for title/buttons, Grid for form fields, Place-style LabelFrames for notes. Validates email format, phone (10 digits), age range (15-80) with error messages.

Q14. Develop a CRUD (Create, Read, Update, Delete) application with Tkinter for managing a simple database-backed model.

Program:

```
import tkinter as tk
from tkinter import messagebox, ttk
import sqlite3

# ■■ Database Setup ■■
conn = sqlite3.connect(":memory:") # use "students.db" for persistent storage
cursor = conn.cursor()
cursor.execute(
    "CREATE TABLE students ("
    "id INTEGER PRIMARY KEY AUTOINCREMENT,"
    "name TEXT NOT NULL,"
    "course TEXT NOT NULL,"
    "marks REAL NOT NULL)"
)
conn.commit()

root = tk.Tk()
root.title("Student CRUD Application")
root.geometry("680x500")
root.config(bg="#e8eaf6")

tk.Label(root, text="Student Management System",
        font=("Arial", 14, "bold"),
        bg="#1a237e", fg="white").pack(fill="x", ipady=8)

# ■■ Input Frame ■■
input_frame = tk.LabelFrame(root, text="Student Details",
                            bg="#e8eaf6", font=("Arial", 10, "bold"))
input_frame.pack(padx=15, pady=8, fill="x")

fields = ["Name", "Course", "Marks"]
entries = {}
for i, f in enumerate(fields):
    tk.Label(input_frame, text=f" {f}:", bg="#e8eaf6",
            font=("Arial", 10)).grid(row=0, column=i*2, padx=5, pady=8)
    e = tk.Entry(input_frame, width=18, font=("Arial", 10))
    e.grid(row=0, column=i*2+1, padx=5)
    entries[f] = e

selected_id = [None]

# ■■ Treeview Table ■■
tree_frame = tk.Frame(root)
tree_frame.pack(padx=15, pady=5, fill="both", expand=True)

scrollbar = tk.Scrollbar(tree_frame)
scrollbar.pack(side=tk.RIGHT, fill=tk.Y)

tree = ttk.Treeview(tree_frame,
```



```

        columns=("ID", "Name", "Course", "Marks"),
        show="headings", yscrollcommand=scrollbar.set, height=10)
for col, w in [("ID", 50), ("Name", 160), ("Course", 180), ("Marks", 100)]:
    tree.heading(col, text=col)
    tree.column(col, width=w, anchor="center")
tree.pack(fill="both", expand=True)
scrollbar.config(command=tree.yview)

def refresh_table():
    for row in tree.get_children():
        tree.delete(row)
    for row in cursor.execute("SELECT * FROM students"):
        tree.insert("", tk.END, values=row)

def create_record():
    n = entries["Name"].get().strip()
    c = entries["Course"].get().strip()
    m = entries["Marks"].get().strip()
    if not (n and c and m):
        messagebox.showwarning("Input Error", "All fields are required!")
        return
    try:
        marks = float(m)
        if not (0 <= marks <= 100):
            raise ValueError
    except ValueError:
        messagebox.showerror("Error", "Marks must be a number between 0-100!")
        return
    cursor.execute("INSERT INTO students (name, course, marks) VALUES (?, ?, ?)",
                   (n, c, marks))
    conn.commit()
    refresh_table()
    clear_entries()
    messagebox.showinfo("Success", f"Student '{n}' added!")

def read_selected(event=None):
    sel = tree.focus()
    if sel:
        vals = tree.item(sel, "values")
        selected_id[0] = vals[0]
        entries["Name"].delete(0, tk.END); entries["Name"].insert(0, vals[1])
        entries["Course"].delete(0, tk.END); entries["Course"].insert(0, vals[2])
        entries["Marks"].delete(0, tk.END); entries["Marks"].insert(0, vals[3])

def update_record():
    if not selected_id[0]:
        messagebox.showwarning("Select", "Please select a record to update!")
        return
    n = entries["Name"].get().strip()
    c = entries["Course"].get().strip()
    m = entries["Marks"].get().strip()
    try:
        marks = float(m)
    except ValueError:
        messagebox.showerror("Error", "Invalid marks!")
        return
    cursor.execute(

```

```

        "UPDATE students SET name=?, course=?, marks=? WHERE id=?",
        (n, c, marks, selected_id[0]))
    conn.commit()
    refresh_table()
    clear_entries()
    messagebox.showinfo("Updated", "Record updated successfully!")

def delete_record():
    if not selected_id[0]:
        messagebox.showwarning("Select", "Please select a record to delete!")
        return
    if messagebox.askyesno("Confirm", "Delete this record?"):
        cursor.execute("DELETE FROM students WHERE id=?", (selected_id[0],))
        conn.commit()
        refresh_table()
        clear_entries()

def clear_entries():
    for e in entries.values():
        e.delete(0, tk.END)
    selected_id[0] = None

tree.bind("<<TreeviewSelect>>", read_selected)

# ■■ CRUD Buttons ■■
btn_frame = tk.Frame(root, bg="#e8eaf6")
btn_frame.pack(pady=8)
for (txt, cmd, col) in [
    ("Add", create_record, "#2e7d32"),
    ("Update", update_record, "#e65100"),
    ("Delete", delete_record, "#b71c1c"),
    ("Clear", clear_entries, "#37474f"),
]:
    tk.Button(btn_frame, text=txt, command=cmd,
              bg=col, fg="white", font=("Arial", 11),
              width=11).pack(side=tk.LEFT, padx=6)

refresh_table()
root.mainloop()

```

Output Description:

➤ **Output: Full CRUD app using SQLite (in-memory). Treeview table shows student records. Add creates new records, click row to auto-fill form, Update saves changes, Delete removes with confirmation. All operations validated.**